МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №3

з дисципліни «Методи та технології інженерії даних»

на тему: «Аналітичний SQL: коректність і продуктивність»

Виконав:
Студент 3 курсу
групи ФЕП-34
Полянський Тимотей

Перевірив:
ас. Фостяк Маркіян Романович

Львів 2026

1. Мета, контекст і середовище

Мета: побудувати перевірений денний звіт за доставленими замовленнями CityFlow, додати накопичувальну суму й останнє замовлення кожного клієнта, а також виміряти вплив індексу на план запиту без зміни результату.

Параметр	Значення
Студент	Полянський Тимотей, група ФЕП-34, `student_id: S08`
СУБД і клієнт	PostgreSQL 16.15 (Docker-образ `postgres:16-alpine`, проєкт `cityflow_s08`), клієнт `psql` 16.15 у контейнері; запуск — `fish scripts/lab03/run_lab03.fish`
Джерело даних	Прийняті таблиці `core` з ЛР №2 (запуск `run_id = 30`, SUCCESS): `core.orders` — 5996 рядків, `core.order_items` — 14027 рядків. Нові дані не завантажувались
Вхідні архіви ЛР №2	`cityflow_S08_base_v1.zip` — SHA-256 `8a8eb377…bede4b661ac` (збігається з наданою сумою); `cityflow_S08_lab02_relational_v1.zip` — `a6f77a3a…cfac046a762`
Період звіту	`date_from = 2025-09-01` (входить), `date_to = 2025-10-01` (не входить) — перший місяць із `delivered`; межі й дні — за часовою зоною `Europe/Kyiv`
Нові об’єкти	Лише схема `lab03_cf`: `settings`, `v_period_orders`, три власні VIEW, копія `bench_*`. Схеми `core`, `stg`, `audit` не змінювались
Репозиторій	github.com/TimoteyPolyanskiy/data-engineering-cityflow-S08, гілка `lab/03-sql-analysis`, Pull Request: pull/3 тег `lab-03-v1` — після перевірки

У вибраному місяці 426 замовлень усіх статусів; у звіт потрапляють лише `delivered`: 342 в UAH і 3 в EUR. Сума `total_amount` — це сума замовлення, а не виторг; валюти не складаються й не конвертуються.

2. Правильність денного звіту

2.1. Помилка після JOIN (блок A)

Валюта	Замовлень	Сума без JOIN	Рядків після JOIN	Сума після JOIN	`SUM(DISTINCT)` після JOIN
UAH	342	194 770.44	767	520 225.15	194 770.44
EUR	3	22.13	6	42.10	22.13

Після JOIN з `core.order_items` кожне замовлення повторюється стільки разів, скільки має позицій, тому його `total_amount` додається кілька разів: в UAH 342 замовлення перетворились на 767 рядків, а сума зросла в 2.67 раза. `SUM(DISTINCT total_amount)` тут випадково дав правильне число: окремий запит показав, що всі 342 суми UAH і 3 суми EUR різні. Цей прийом прибирає однакові **значення**, а не повторні `order_id`, тож два різні замовлення з однаковою сумою були б втрачені — це не виправлення JOIN.

2.2. Ручний розрахунок на власних замовленнях

Заклад `S08-M00038`, дата 2025-09-01, валюта UAH, обидва замовлення `delivered`, кожне має кілька позицій:

`order_id`	`total_amount`	Позицій	`SUM(quantity)`
S08-O0000006	752.24	3	4
S08-O0000007	506.83	3	3

Правильна сума: 752.24 + 506.83 = **1259.07 UAH**, 2 замовлення, 7 одиниць. Після JOIN кожна сума повторюється за кожною з трьох позицій: 3 × 752.24 + 3 × 506.83 = **3777.21 UAH** (утричі більше). Рядок денного звіту за цей день і заклад: `order_count = 2`, `order_amount = 1259.07`, `avg_order_amount = 629.54`, `accepted_units = 7`, `orders_without_items = 0` — збігається з ручним розрахунком.

2.3. VIEW `lab03_cf.v_daily_orders` і перші 10 рядків

Три CTE: `selected_orders` — замовлення місяця зі статусом `delivered` (рівень «одне замовлення»); `items_by_order` — позиції, зведені до одного рядка на `order_id` (`SUM(quantity) AS units`); `orders_with_units` — `LEFT JOIN` цих двох наборів за `order_id` з `COALESCE(units, 0)` та ознакою `i.order_id IS NULL`. Оскільки в `items_by_order` не більше одного рядка на замовлення, з’єднання 1:1 нічого не множить, а LEFT JOIN не

губить замовлень без позицій. Фінальний SELECT групує за `report_date, merchant_id, currency`; середнє — `ROUND(SUM(total_amount) / COUNT(*), 2)`.

report_date	merchant_id	Валюта	Замовл.	Сума	Середнє	Одиниць	Без позицій
2025-09-01	S08-M00001	UAH	1	256.31	256.31	1	0
2025-09-01	S08-M00004	UAH	2	391.37	195.69	4	0
2025-09-01	S08-M00005	UAH	1	761.30	761.30	2	0
2025-09-01	S08-M00020	UAH	1	346.96	346.96	1	0
2025-09-01	S08-M00022	UAH	1	188.82	188.82	2	0
2025-09-01	S08-M00023	UAH	1	874.36	874.36	4	0
2025-09-01	S08-M00027	UAH	1	145.61	145.61	1	0
2025-09-01	S08-M00032	UAH	1	219.13	219.13	1	0
2025-09-01	S08-M00038	UAH	2	1259.07	629.54	7	0
2025-09-02	S08-M00002	UAH	1	720.93	720.93	3	0

Усього у звіті 284 денні рядки, 345 замовлень і 924 одиниці; замовлень без позицій у цьому місяці немає (0), але LEFT JOIN залишено, щоб такі замовлення не зникали на інших даних.

2.4. Перевірка `02_check_daily.sql`

`daily_exact_match`: `difference_rows = 0`, **PASS** — звіт збігається з незалежним еталоном рядок у рядок (`EXCEPT ALL` в обидва боки).

Валюта	Замовлень у джерелі	Замовлень у звіті	Сума в джерелі	Сума у звіті	Різниця к-сті	Різниця суми
EUR	3	3	22.13	22.13	0	0.00
UAH	342	342	194 770.44	194 770.44	0	0.00

3. Віконні функції

3.1. Накопичувальна сума `v_running_orders`

`SUM(order_amount) OVER (PARTITION BY merchant_id, currency ORDER BY report_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`. Приклад — заклад `S08-M00016`, UAH (17 днів із доставленими замовленнями; пропущені дні не додаються):

report_date	order_amount	cumulative_amount	report_date	order_amount	cumulative_amount
2025-09-02	629.64	629.64	2025-09-20	1148.22	10 036.87
2025-09-03	550.94	1180.58	2025-09-24	477.79	10 514.66
2025-09-05	615.24	1795.82	2025-09-25	979.11	11 493.77
2025-09-06	736.78	2532.60	2025-09-26	549.65	12 043.42
2025-09-08	276.07	2808.67	2025-09-28	698.33	**12 741.75**

Останнє `cumulative_amount = 12 741.75` дорівнює `SUM(order_amount)` цієї групи (окремий запит: 12 741.75). Кількість рядків не змінилась: `daily_rows = 284`, `running_rows = 284`.

3.2. Останнє замовлення клієнта `v_latest_customer_order`

`ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY created_at DESC, order_id DESC)` у CTE, у зовнішньому SELECT — `rn = 1`; джерело — усі статуси місяця. Клієнт `S08-C000352` має 5 замовлень (час — UTC):

order_id	created_at	Статус	Валюта	Сума	rn
S08-O0000411	**2025-09-29 13:35:42+00**	**delivered**	**UAH**	**1712.60**	**1**
S08-O0000329	2025-09-24 13:35:28+00	delivered	UAH	294.37	2
S08-O0000243	2025-09-17 20:52:35+00	delivered	UAH	955.91	3
S08-O0000170	2025-09-12 14:29:44+00	delivered	UAH	2303.64	4
S08-O0000141	2025-09-10 12:06:54+00	delivered	UAH	868.77	5

Обрано `S08-O0000411` (заклад `S08-M00033`), бо в нього найпізніший `created_at`; `order_id` знадобився б лише за однакового часу, а таких збігів у місяці немає (0 клієнтів). Це останнє **створене** замовлення, а не остання зміна статусу: історії подій у `core` немає. Перевірка `04_check_windows.sql`: `running_exact_match` — 0 різниць, **PASS**; `latest_order_exact_match` — 0 різниць, **PASS**; `expected_customers = 294`, `actual_rows = 294`.

4. Продуктивність: план запиту та індекс

Копія: `05_bench_prepare.sql` скопіював у `lab03_cf.bench_orders` 5996 рядків `core.orders` без індексів (`pg_indexes` — 0 рядків). **Фільтри** (`bench_parameters`): заклад `S08-M00016` (найбільше замовлень у місяці), `created_at` від `2025-08-31 21:00:00+00` до `2025-09-30 21:00:00+00` (не включно) — це місяць за Києвом; відібрано **29 рядків** усіх статусів. Після `07_bench_index.sql` є один індекс `idx_lab03_cf_bench_merchant_created (merchant_id, created_at)`. Кожен стан: 1 прогрів (не враховано) і 3 вимірювання `Execution Time`.

Умови	Час 1, мс	Час 2, мс	Час 3, мс	Медіана, мс	Прогрів, мс
Без індексу	0.359	0.684	0.638	**0.638**	0.355
З індексом	0.097	0.086	0.088	**0.088**	0.093

План без індексу (`docs/lab03/plan_before.txt`, третє вимірювання):

```
 Seq Scan on bench_orders  (cost=62.10..217.03 rows=1 width=27) (actual time=0.039..0.548 rows=29 loops=1)
   Filter: ((created_at >= $1) AND (created_at < $2) AND (merchant_id = $0))
   Rows Removed by Filter: 5967
   Buffers: shared hit=53
   InitPlan 1 (returns $0)
     ->  Seq Scan on bench_parameters  (cost=0.00..20.70 rows=1070 width=32) (actual time=0.001..0.002 rows=1 loops=1)
           Buffers: shared hit=1
   InitPlan 2 (returns $1)
     ->  Seq Scan on bench_parameters bench_parameters_1  (cost=0.00..20.70 rows=1070 width=8) (actual time=0.011..0.011 rows=1
loops=1)
           Buffers: shared hit=1
   InitPlan 3 (returns $2)
     ->  Seq Scan on bench_parameters bench_parameters_2  (cost=0.00..20.70 rows=1070 width=8) (actual time=0.002..0.002 rows=1
loops=1)
           Buffers: shared hit=1
 Planning:
   Buffers: shared hit=46
 Planning Time: 0.508 ms
 Execution Time: 0.638 ms
(17 rows)
```

План з індексом (`docs/lab03/plan_after.txt`):

```
 Index Scan using idx_lab03_cf_bench_merchant_created on bench_orders  (cost=62.38..70.41 rows=1 width=27) (actual time=0.027..0.033
rows=29 loops=1)
   Index Cond: ((merchant_id = $0) AND (created_at >= $1) AND (created_at < $2))
   Buffers: shared hit=9
   InitPlan 1 (returns $0)
     ->  Seq Scan on bench_parameters  (cost=0.00..20.70 rows=1070 width=32) (actual time=0.003..0.003 rows=1 loops=1)
           Buffers: shared hit=1
   InitPlan 2 (returns $1)
     ->  Seq Scan on bench_parameters bench_parameters_1  (cost=0.00..20.70 rows=1070 width=8) (actual time=0.000..0.001 rows=1
loops=1)
           Buffers: shared hit=1
   InitPlan 3 (returns $2)
     ->  Seq Scan on bench_parameters bench_parameters_2  (cost=0.00..20.70 rows=1070 width=8) (actual time=0.000..0.000 rows=1
loops=1)
           Buffers: shared hit=1
 Planning:
   Buffers: shared hit=95
 Planning Time: 0.376 ms
 Execution Time: 0.088 ms
(16 rows)
```

Перевірка `08_bench_check.sql`: `benchmark_same_result`, `difference_rows = 0`, **PASS** — після індексу запит повертає ті самі 29 рядків.

Пояснення. Без індексу PostgreSQL читав усю копію послідовно (`Seq Scan`): переглянув 5996 рядків, 5967 відкинув фільтром і торкнувся 53 блоків. З індексом план змінився на `Index Scan` за `idx_lab03_cf_bench_merchant_created`: умова `Index Cond` охоплює і `merchant_id`, і діапазон `created_at`, тому прочитано лише 9 блоків. Медіана часу зменшилась з 0.638 до 0.088 мс, але це частки мілісекунди, а розкид без індексу (0.359–0.684 мс) завеликий, щоб упевнено називати коефіцієнт прискорення. Усі блоки в обох випадках уже були в буфері (`shared hit`, без `read`), тож порівнюється робота процесора, а не диска. Результат правдоподібний: таблиця мала (53 сторінки), а запит відбирає близько 0.5 % рядків — індекс за «рівність, потім діапазон» точно відповідає такому фільтру. Оцінка `rows=1` проти фактичних 29 пояснюється

тим, що значення фільтра надходять із підзапитів (`InitPlan`) і невідомі планувальнику під час планування. Висновок: для цього запиту індекс доречний за кількістю прочитаних блоків, але на нашому обсязі виграш у часі практично непомітний, а індекс займає місце й сповільнює запис.

5. Висновки

1. JOIN замовлень із позиціями множить суми: в UAH 342 доставлені замовлення дали 767 рядків, і сума зросла з 194 770.44 до 520 225.15. Попереднє агрегування позицій до рівня замовлення та LEFT JOIN дають правильний звіт: 284 денні рядки, звірка окремо для UAH і EUR — 0 різниць, `daily_exact_match` — PASS.
2. Віконні функції не згортають рядки: `v_running_orders` має ті самі 284 рядки, а накопичення для `S08-M00016` (UAH) закінчується на 12 741.75 — сумі групи. `v_latest_customer_order` дає рівно один рядок на кожного з 294 клієнтів місяця; обидві перевірки — PASS.
3. Індекс `(merchant_id, created_at)` змінив `Seq Scan` на `Index Scan`, зменшив кількість блоків з 53 до 9 і медіану часу з 0.638 до 0.088 мс без зміни результату (PASS). На копії з 5996 рядків різниця становить частки мілісекунди, тому користь індексу тут — радше у прочитаних блоках, ніж у помітному часі.

6. Використання ШІ

Інструмент: Claude. Поставлені завдання: підготувати чернетки трьох VIEW (`v_daily_orders` через три CTE, `v_running_orders`, `v_latest_customer_order`) і скрипт запуску файлів комплекту з вимірюваннями; пояснити рядки `EXPLAIN (ANALYZE, BUFFERS)`. Внесок автора: постановка завдань, перевірка результатів запусків і пояснення запитів. Перевірка на власних даних S08: готові перевірки викладача дали PASS (`daily_exact_match`, `running_exact_match`, `latest_order_exact_match`, `benchmark_same_result`), суми звірено окремо для UAH та EUR, ручний розрахунок двох замовлень збігся з рядком денного звіту. Цікавий випадок: `SUM(DISTINCT total_amount)` дав правильну суму, але лише тому, що всі 345 сум доставлених замовлень місяця різні; це перевірено окремим запитом, тому такий запит не вважається виправленням JOIN. Виправлена помилка: у першій версії скрипта запуску функції fish посилалися на локальну змінну шляху, якої функції не бачать; помилку помічено до запуску, шляхи замінено на відносні.

7. Контрольні запитання

1. **Чому SUM(total_amount) після JOIN може бути неправильним?** `orders` має рівень «одне замовлення», а `order_items` — «одна позиція». JOIN повторює рядок замовлення для кожної позиції, і його `total_amount` додається кілька разів: у нашому прикладі 1259.07 перетворилось на 3777.21 UAH. INNER JOIN, крім того, прибрав би замовлення без позицій.
2. **Як попереднє агрегування змінює grain і навіщо LEFT JOIN?** CTE `items_by_order` зводить позиції до рівня «одне замовлення» (один рядок на `order_id`), тому з'єднання з замовленнями стає 1:1 і нічого не множить. LEFT JOIN зберігає замовлення, які не мають прийнятих позицій; для них `units = 0`, а `i.order_id IS NULL` дає лічильник `orders_without_items`.
3. **Чим GROUP BY відрізняється від window function?** GROUP BY згортає групу в один рядок. Віконна функція рахує значення над пов'язаними рядками, але залишає кожен рядок: у `v_running_orders` так само 284 рядки, як у денному звіті.
4. **Навіщо у running total окремі merchant_id і currency?** Щоб накопичення не змішувало різні заклади та різні валюти: суми в UAH і EUR не можна додавати, а підсумок одного закладу не має «перетікати» в інший. `PARTITION BY merchant_id, currency` починає нове накопичення для кожної такої пари.
5. **Що саме визначає ROW_NUMBER у вашому запиті, а чого він не доводить?** Він нумерує замовлення кожного клієнта від найпізніше створеного (`created_at DESC`, за однакового часу — `order_id DESC`), і `rn = 1` — останнє створене замовлення місяця. Він не доводить, що це остання зміна статусу чи «нова версія» замовлення: історії подій у `core` немає.
6. **Чим EXPLAIN відрізняється від EXPLAIN ANALYZE?** EXPLAIN лише показує план і оцінки планувальника (`cost`, `rows`), не виконуючи запит. EXPLAIN ANALYZE реально виконує запит і додає фактичні рядки, час і (з `BUFFERS`) прочитані блоки — тому його запускаємо лише для SELECT.
7. **Чому індекс може не допомогти і як перевірити однаковість результату?** На малій таблиці послідовне читання кількох сторінок уже дуже швидке, а для неселективного фільтра планувальник обере `Seq Scan`; індекс ще й займає місце й сповільнює запис. Однаковість перевіряємо порівнянням наборів рядків до й після (`EXCEPT ALL` в обидва боки, `08_bench_check.sql`): 0 різниць — PASS.